AgendaFlow: Task-Adaptive Meeting Protocols for Multi-Agent LLM Reasoning

Anonymous ACL submission

Abstract

Multi-agent large language model (LLM) systems often rely on independent solvers, critique, debate, or answer aggregation. While these designs increase diversity and feedback, they do not necessarily create effective group reasoning: information may be poorly shared, expertise underused, discussions may drift, and consensus may emerge before unresolved issues are examined. We introduce AGENDAFLOW, a meeting-science-inspired framework that treats multi-agent reasoning as a structured meeting rather than parallel answering or generic debate. AGENDAFLOW routes each task to a suitable agenda, instantiates complementary functional roles, and uses a moderator to manage phases, speaking order, shared artifacts, unresolved issues, and completion checks. This enables agents to pool information, surface constraints, compare alternatives, repair errors, and converge through a managed collective decision process. Across five reasoning, planning, and rule- or evidence-based benchmarks, AGENDAFLOW consistently outperforms single-agent, sampling-based, debate-style, and multi-agent orchestration baselines. Ablations, token-matched comparisons, and process-level analyses suggest that the gains come from meeting-level organization rather than agent diversity or additional computation alone, highlighting meeting design as a missing dimension in multi-agent LLM systems. Anonymous code and data are available at: https://anonymous.4open.science/r/MISP-64CF/.

1 Introduction

Large language models (LLMs) have achieved strong performance on a wide range of reasoning, planning, and decision-making tasks (Wei et al., 2022; Wang et al., 2022; Zhou et al., 2022). However, complex tasks remain challenging because they often require the model to identify goals, track constraints, compare alternatives, detect inconsistencies, and revise intermediate conclusions (Yao et al., 2022; Wu et al., 2023; Li et al., 2023a) rather than producing a plausible answer in a single reasoning trace (Yao et al., 2023; Madaan et al., 2023; Shinn et al., 2023). When these process requirements are not explicitly managed, LLMs may omit critical constraints, rely on unsupported assumptions, overlook alternative solutions, or converge prematurely on an incorrect answer (Du et al., 2023; Chen et al., 2024).

Multi-agent methods have emerged as a promising direction for improving LLM reasoning (Chen et al., 2023b; Xu et al., 2023). Existing approaches elicit multiple reasoning paths, aggregate independently generated answers (Jiang et al., 2023; Li et al., 2024a), or allow several agents to critique one another through debate (Chan et al., 2023; Liang et al., 2023). These methods suggest that multiple agents can provide useful diversity and error-checking signals (Hong et al., 2023; Qian et al., 2023). Nevertheless, many existing systems treat multi-agent interaction primarily as a mechanism for generating more candidate answers or more adversarial feedback. The structure of the interaction itself is often left relatively underdesigned (Li et al., 2024b; Liu et al., 2024), as illustrated in Figure 1. This limitation is especially important because complex reasoning tasks are not homogeneous: planning tasks require explicit treatment of goals, constraints, conflicts, and feasibility; verification tasks require standards, evidence, risks, and counterarguments; decision-selection tasks require criteria, trade-offs, hard constraints, and convergence. Applying the same generic debate template across such tasks may lead to repeated arguments, superficial disagreement, neglected constraints, or premature consensus (Guo et al., 2024; Chen et al., 2023a).

We argue that multi-agent LLM systems should be studied as structured deliberative processes. This perspective is inspired by meeting science,

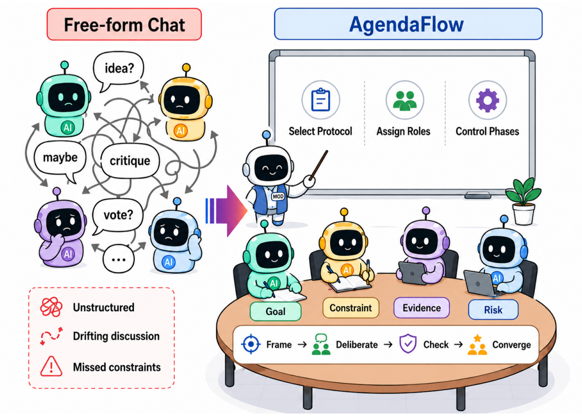

Figure 1: Motivation of AGENDAFLOW. Free-form multi-agent chat can produce unstructured and drifting discussions, while AGENDAFLOW organizes reasoning through task-adaptive protocol selection, role assignment, and phase control.

which studies how groups coordinate attention, distribute expertise, manage discussion, and transform dispersed information into collective decisions (Allen et al., 2015; Mroz et al., 2018). Effective meetings are not merely open-ended conversations; they depend on clear objectives, appropriate participant composition, agenda structure, facilitation, turn allocation, and explicit mechanisms for surfacing unresolved issues (Kauffeld and Lehmann-Willenbrock, 2012; Nixon and Littlepage, 1992). These process-level principles provide a useful foundation for designing how LLM agents reason together on complex tasks.

To this end, we introduce AGENDAFLOW, a task-adaptive framework that treats multi-agent LLM reasoning as a structured meeting process. Rather than asking agents to solve the whole task independently or debate over final answers, AGENDAFLOW organizes them to progressively construct a shared deliberation state. For each input, it selects an agenda with task-specific phases, assigns agents to complementary functional roles, and uses a moderator to manage information sharing, unresolved issues, phase transitions, and completion checks. Each phase produces an intermediate artifact that is carried forward to the next phase, enabling agents to pool evidence, examine constraints, compare alternatives, and converge in a disciplined manner. Our central hypothesis is that multi-agent reasoning benefits not merely from more agents, but from more interaction rounds and protocols that make knowledge sharing and collective deliberation effective.

We evaluate AGENDAFLOW across multiple benchmarks covering reasoning, planning, and evidence-and-rule verification. AGENDAFLOW improves over representative single-agent prompting, sampling-based aggregation, and multi-agent collaboration methods. Further analysis shows that AGENDAFLOW improves process-level qualities such as constraint utilization, error detection, repair success, perspective coverage, and reduced premature consensus.

Our contributions are threefold:

- We identify a key limitation of existing multi-agent LLM systems: many simulate discussion, debate, or answer aggregation, but do not explicitly model the meeting process through which groups coordinate attention, share information, manage disagreement, and reach decisions.
- We propose AGENDAFLOW, a meeting-science-inspired framework that operationalizes agenda design, functional participant roles, facilitation, shared artifacts, unresolved-issue tracking, and closure checks for multi-agent LLM reasoning.
- We show across five benchmarks that meeting-structured agent interaction improves both final task performance and reasoning-process quality, with ablations and token-matched comparisons indicating that the improvement is not explained by more agents, more turns, or more computation alone.

2 Related Work

Large Language Models and Reasoning. Large language models have become a general foundation for solving complex language and reasoning tasks through prompting (Brown et al., 2020; Cobbe et al., 2021; Suzgun et al., 2022). A major line of work improves reasoning without parameter updates by eliciting intermediate computation, rationales, or structured inference traces, including chain-of-thought prompting and zero-shot reasoning (Nye et al., 2021; Kojima et al., 2022; Press et al., 2022; Wei et al., 2022). Subsequent methods extend this paradigm through diverse reasoning paths, decomposition, planning, search-based exploration, tool use, and iterative self-refinement (Wang et al., 2022; Khot et al., 2022; Wang et al., 2023; Besta et al., 2024; Gao et al., 2023; Schick et al., 2023; Madaan et al., 2023). Together, these studies show that structuring the reasoning process can improve LLM reliability on challenging tasks.

Multi-Agent Debate and Collaboration. Multi-agent approaches organize multiple LLM-based agents to solve problems through interaction, simulation, and coordinated decision-making (Park et al., 2023; Wang et al., 2024; Li et al., 2023b). Prior work has explored collaborative discussion, role-based deliberation, adversarial debate, round-table consensus, and multi-agent evaluation (Du et al., 2023; Liang et al., 2023; Chen et al., 2024; Zhou et al., 2023; Liu et al., 2023). In these systems, agents may propose, critique, revise, or aggregate answers through voting, judging, or confidence-based mechanisms (Chan et al., 2023; Xu et al., 2023; Jiang et al., 2023). Recent frameworks further investigate task decomposition, role assignment, communication protocols, and coordinated execution in broader collaborative environments (Xie et al., 2023; Gao et al., 2024; Liu et al., 2024). This line of work demonstrates that complementary perspectives and iterative interaction can improve the robustness of LLM reasoning.

Meeting Science and Structured Deliberation. Meeting science studies collective work as organized deliberation rather than conversation alone, focusing on how groups pool information, sustain participation, make decisions, and carry outcomes forward (Schwartzman, 1989; Rogelberg et al., 2006; Leach et al., 2009; Lehmann-Willenbrock et al., 2013). It provides conceptual grounding for viewing multi-participant reasoning as a structured process of information sharing, evaluation, and closure (Stasser and Titus, 1985; Mesmer-Magnus and DeChurch, 2009; Wittenbaum et al., 2004; Okhuysen and Eisenhardt, 2002; De Dreu and West, 2001; Hirokawa, 1985; Poole and Roth, 1989; Woolley et al., 2010).

3 Method

3.1 Overview

We propose AgendaFlow, a structured framework for complex multi-agent LLM reasoning. Given an input task x, AgendaFlow first identifies its reasoning objective, such as planning, problem solving, verification, or decision selection, and then instantiates a corresponding task-adaptive meeting protocol to guide the discussion process.

As shown in Figure 2, AgendaFlow consists of three core components. First, the task-adaptive protocol module selects a reasoning flow P_τ according to the inferred task type τ. Each protocol specifies a sequence of phases, where every phase is associated with a concrete reasoning objective. Second, the functional role coverage module assigns agents to complementary reasoning functions required by the selected protocol, such as constraint checking, evidence inspection, risk analysis, or feasibility verification. Third, the moderator-governed phase control module regulates the discussion by determining speaking order, enforcing phase boundaries, checking whether the current phase has satisfied its completion condition, and deciding whether to advance, repeat, or terminate the reasoning process. The final output is generated after the moderator determines that the required phases have been completed and the remaining unresolved issues have been sufficiently addressed. Algorithm 1 details the phase-level execution procedure.

3.2 Task-Adaptive Protocols

AgendaFlow implements task adaptation through a protocol library. Each protocol is constructed by translating meeting-science and organizational-collaboration theories into a reasoning template. For each task type, we identify its characteristic process risks, such as premature convergence, criteria drift, insufficient evidence use, unstructured disagreement, or weak closure, and map them into protocol elements including phase ordering, intermediate artifacts, role activation, and completion criteria.

Formally, a protocol template is represented as

$$P_\tau = \{(\phi_t, o_t, a_t, \mathcal{A}_t, c_t)\}_{t=1}^{T},$$

where τ denotes the protocol type, ϕ_t is the t-th reasoning phase, o_t is the phase objective, a_t is the expected intermediate artifact, $\mathcal{A}_t$ is the subset of roles activated in the phase, and c_t is the completion criterion. Thus, a protocol specifies not only what agents should discuss, but also what artifact must be produced before the reasoning process advances.

We use the problem-solving protocol as a representative example. This protocol is selected when the task concerns an observed failure, anomaly, or deviation that requires diagnosis and corrective action. Following structured problem-solving principles, it organizes reasoning into four stages: problem framing, diagnostic inquiry, intervention design, and answer convergence. These stages require agents to clarify the problem boundary, reconstruct relevant facts, test causal hypotheses, map interventions to diagnosed causes, and finally synthesize a response that satisfies the original task objective.

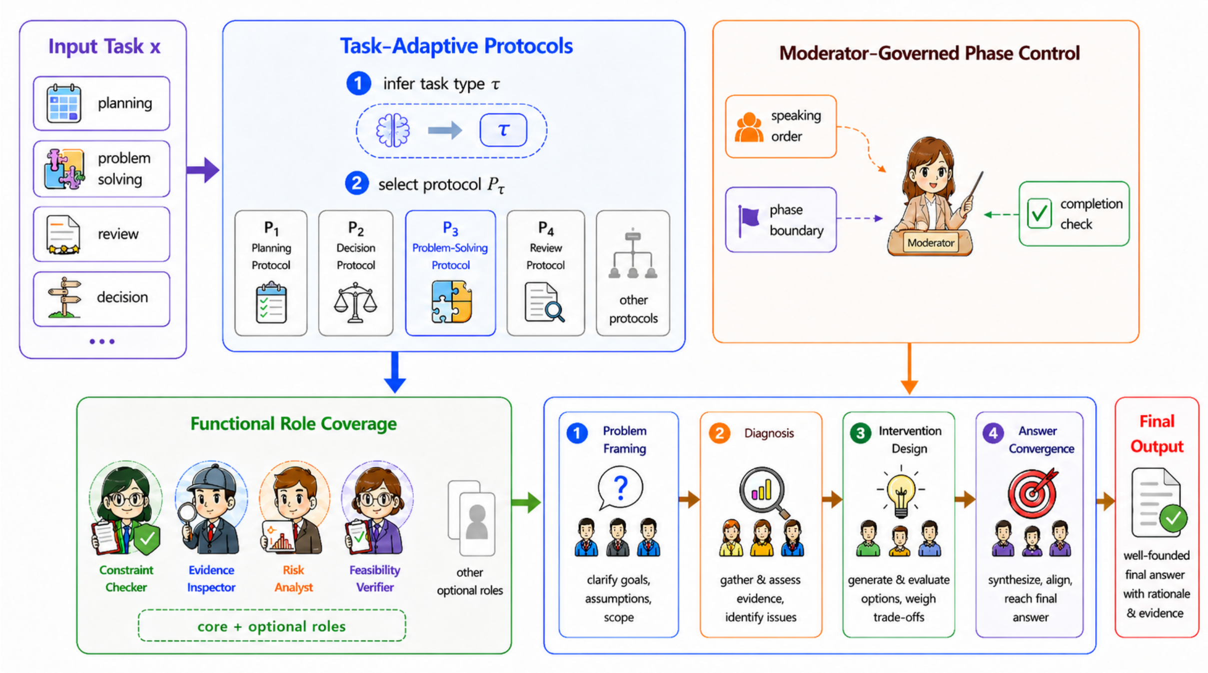

Figure 2: Overview of AGENDAFLOW. AGENDAFLOW selects a task-adaptive meeting protocol, instantiates functionally complementary roles, and uses a moderator to control phase execution through speaking order, phase boundaries, and completion checks.

3.3 Functional Role Coverage

For each protocol type τ, AGENDAFLOW defines a role-slot schema

$$\mathcal{S}_\tau = \mathcal{S}_\tau^{\text{core}} \cup \mathcal{S}_\tau^{\text{opt}},$$

where $\mathcal{S}_\tau^{\text{core}}$ contains mandatory role slots and $\mathcal{S}_\tau^{\text{opt}}$ contains optional role slots. Core slots specify the reasoning functions that must appear in the protocol-guided reasoning process, while optional slots provide additional candidate perspectives that can be selected according to the task context.

After the protocol is selected, AGENDAFLOW instantiates the selected slots into task-specific agent profiles. Each instantiated role keeps its underlying functional slot, but receives a concrete participant identity and behavioral description for the current task:

$$\mathcal{R}_\tau = \{(s_i, title_i, b_i, \theta_i, u_i)\}_{i=1}^{|\mathcal{R}_\tau|},$$

where $s_i \in \mathcal{S}_\tau$ is an internal slot identifier, $title_i$ is the participant identity used in the multi-agent discussion, and b_i, θ_i, and u_i denote the participant background, thinking style, and unique value, respectively. The slot identifier preserves the role's functional position in the selected protocol, while the generated participant identity adapts that role to the specific task. As a result, different task types can activate different combinations of roles, while still preserving the protocol-level structure needed for organized multi-agent LLM reasoning.

3.4 Moderator-Governed Phase Control

AGENDAFLOW uses a moderator agent to execute the selected protocol. The moderator is not treated as a domain participant; its role is to manage the reasoning state, coordinate role activation, enforce phase boundaries, and determine whether the discussion should advance, repeat, or terminate.

At each phase, the moderator maintains a phase-level reasoning state:

$$z_t = (x, \phi_t, H_{<t}, U_t),$$

where x is the input task, ϕ_t is the current phase, $H_{<t}$ denotes the reasoning history before phase t, and U_t denotes the unresolved issues identified so far.

Based on this state, the moderator issues a phase instruction and invites the agents activated for the current phase to contribute. Each activated agent responds according to its role specification, the phase objective, and previous phase artifacts. After collecting the responses, the moderator synthesizes them into a phase artifact and updates the unresolved issue set. The moderator then checks whether the phase artifact satisfies the corresponding completion criterion. If the criterion is satisfied, the reasoning process advances to the next phase. If required information is missing or unresolved issues remain, the moderator may repeat the phase within the available interaction budget or carry unresolved issues forward with a targeted instruction to the relevant roles. The process terminates when all protocol phases are completed. The final output is produced from the completed phase artifacts.

4 Experiments

4.1 Tasks

We evaluate AGENDAFLOW on five complex reasoning benchmarks that require models to reason over actions, states, rules, evidence, or constraints. ACPBench (Kokel et al., 2025) evaluates action-centered planning and reasoning over state changes, including preconditions, effects, and plan validity. ACPBench Hard (Kokel et al., 2026) extends this setting with more demanding instances that require longer reasoning chains and stricter constraint tracking. BoardgameQA (Kazemi et al., 2023) focuses on rule-based question answering in board-game scenarios, where models must interpret rules and track evolving states under ambiguity or contradiction. MuSR (Sprague et al., 2024) evaluates multi-step soft reasoning over narratives, requiring evidence aggregation and commonsense inference across dispersed information. Natural Plan (Zheng et al., 2024) assesses everyday planning scenarios, such as travel, meetings, and scheduling, where models must balance goals, constraints, and feasibility.

4.2 Baselines

We compare AGENDAFLOW with representative single-agent, sampling-based, and multi-agent reasoning methods. Direct Answering prompts the backbone model to output an answer without explicit intermediate reasoning. Chain-of-Thought (Wei et al., 2022) elicits a step-by-step reasoning trace before prediction, while Self-Consistency (Wang et al., 2022) samples multiple such traces and aggregates their answers. Vanilla Multi-Agent Debate (Du et al., 2023) uses a fixed round-based protocol in which agents propose, critique, and revise answers. AutoGen (Wu et al., 2023) is a general conversational multi-agent framework with role-conditioned agents and coordinator-guided turn allocation. ReConcile (Chen et al., 2024) organizes multiple agents in a round-table discussion and aggregates their conclusions through consensus. MAgICoRe (Chen et al., 2025) performs iterative coarse-to-fine refinement across multiple agents. DyLAN (Liu et al., 2024) dynamically selects agents and communication structures during inference.

4.3 Implementation Details

All experiments are conducted in a zero-shot setting using Kimi2 as the backbone model. We evaluate MuSR on the full benchmark. Since the original pools of the other four benchmarks are substantially larger, we construct an evaluation set of 1,200 examples for each benchmark to control evaluation cost. The examples are selected by stratified random sampling according to the task categories within each benchmark. For each benchmark, we use its official prompt, parser, and scorer for generation, answer extraction, and evaluation. The decoding temperature is set to 0.2 for all methods, while all other decoding parameters are kept at their default values. We repeat each experimental setting five times with independent random seeds and report the mean score. Standard deviations are provided in Appendix C. Self-consistency uses 10 independent samples per instance. For all multi-agent methods, we instantiate four agents and set the interaction budget to four rounds.

We further assess AGENDAFLOW against the strongest non-AGENDAFLOW baseline on each benchmark using paired tests over per-instance correctness averaged across seeds. We compute 95% paired-bootstrap confidence intervals and paired-randomization p-values, with Holm-Bonferroni correction applied across the five benchmark-level comparisons. All five benchmark-level gains remain statistically significant after correction; detailed results are provided in Appendix C.2.

4.4 Main Results

Table 1 reports the main results across the five benchmark-level evaluations. AGENDAFLOW achieves the highest score on all benchmark-level averages, obtaining an overall average of 64.72%. Compared with the strongest baseline, MAgICoRe, AGENDAFLOW improves by 2.12 absolute points. The gains are most pronounced on ACPBench and ACP-Hard, where it exceeds the best baseline by 2.43 and 3.62 points, respectively. This indicates that structured agent coordination is particularly beneficial for tasks that require maintaining action

Method	ACPBench	ACP-Hard	BoardgameQA	MuSR				Natural Plan				Avg.
				Murder	Object	Team	Avg.	Calendar	Meeting	Trip	Avg.	
Direct	58.42	35.08	73.00	66.40	50.78	63.60	60.26	34.23	35.74	31.27	33.75	52.10
CoT	57.83	36.50	74.58	81.20	61.72	78.00	73.64	49.85	**50.75**	12.73	37.78	56.07
Self-Consistency	59.08	36.83	75.58	82.30	62.90	78.81	74.67	46.20	47.20	11.36	34.92	56.22
MAD	67.33	44.25	61.58	58.80	44.14	67.20	56.71	60.69	42.94	<u>32.40</u>	45.34	55.04
AutoGen	65.50	42.83	68.25	70.10	54.10	67.80	64.00	54.80	39.40	27.54	40.58	56.23
ReConcile	68.42	45.17	<u>76.10</u>	82.05	63.05	78.95	74.68	58.20	47.65	31.85	45.90	62.05
MAgICoRe	68.92	45.75	76.05	<u>82.35</u>	<u>63.18</u>	<u>79.05</u>	<u>74.86</u>	61.10	<u>49.20</u>	31.90	<u>47.40</u>	<u>62.60</u>
DyLAN	<u>69.15</u>	<u>46.05</u>	75.40	79.60	61.05	76.95	72.53	<u>61.35</u>	46.80	31.75	46.63	61.95
AGENDAFLOW	**71.58**	**49.67**	**77.25**	**83.60**	**64.70**	**80.54**	**76.28**	**63.80**	49.10	**33.59**	**48.83**	**64.72**

Table 1: Main results across five benchmarks. All values are mean accuracy scores in percentage over five independent runs. For MuSR and Natural Plan, we report both subtask-level results and the benchmark-level average. The final average is computed over the five benchmark-level scores. Standard deviations are reported in Appendix C. Best results are in bold and second-best results are underlined.

states, constraints, and intermediate commitments over multiple reasoning steps.

AGENDAFLOW also improves over the strongest baselines on BoardgameQA, MuSR, and Natural Plan, with gains of 1.15, 1.42, and 1.43 points, respectively. These improvements are smaller but consistent across benchmarks that differ in task format and reasoning structure. At the subtask level, AGENDAFLOW obtains the best result on all reported categories except meeting planning, where chain-of-thought prompting performs best and MAgICoRe is slightly ahead of AGENDAFLOW. Overall, the results suggest that task-adaptive protocols provide benefits beyond independent sampling, debate-style interaction, and generic multi-agent coordination.

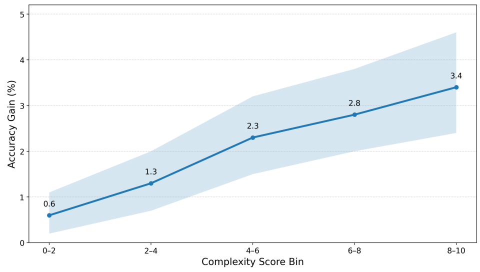

Figure 3: AGENDAFLOW gain over the strongest non-AGENDAFLOW baseline across complexity-score bins. The gain is computed within each bin and averaged over benchmarks. Gains increase with complexity, suggesting that structured reasoning is most beneficial for harder instances.

4.5 Difficulty-Stratified Analysis

We further examine whether the benefit of AGENDAFLOW varies with instance complexity. Since the benchmarks do not share a unified difficulty annotation scheme, we construct a computable complexity score for each instance. The score is based on benchmark-specific features that reflect the amount of information to track and the length of reasoning required. For Natural Plan, we use the number of constraints, candidate locations or time slots, and planning steps. For ACPBench and ACPBench Hard, we use the number of state variables, actions, and reasoning steps. For MuSR, we use text length, the number of entities, and the number of evidence spans. For BoardgameQA, we use the number of turns, rules, and state transitions. The resulting features are normalized into a score from 0 to 10.

Figure 3 reports the accuracy gain of AGENDAFLOW over the strongest non-AGENDAFLOW baseline across equal-width complexity-score bins. The gain increases as the complexity score grows. This trend reduces the dependence on manually chosen difficulty thresholds and suggests that the advantage of AGENDAFLOW is associated with problem complexity rather than a particular discrete partition. In higher-complexity bins, the gain becomes larger, suggesting that protocol-guided reasoning is more beneficial when reasoning requires sustained constraint tracking, intermediate-state verification, and revision.

4.6 Ablation Study

To isolate the contribution of different design choices in AGENDAFLOW, we compare the full framework with five variants. Generic-MA removes the protocol library, functional role slots, and moderator control while keeping the same backbone model, number of agents, and interaction rounds. Indep.-Role uses the same instantiated agents and role profiles as AGENDAFLOW, but each agent independently solves the full task without observing other agents or participating in phase-level discussion; w/o Role Theory removes the

Method	ACP-H	BGQA	MuSR	NP	Avg.
AGENDAFLOW	**48.67**	**76.25**	**75.28**	**47.83**	**62.01**
Generic	44.75	70.17	67.22	42.42	56.14
– Role	46.92	75.83	74.83	46.25	60.96
– Routing	45.83	74.67	73.94	44.75	59.80
Indep. Role	45.60	73.40	72.40	44.20	58.90
– Mod.	43.17	71.58	69.89	41.92	56.64

Table 2: Ablation results on four benchmarks. All values are mean accuracy scores over five independent runs. ACP-H, BGQA, NP, and Mod. denote ACP-Bench Hard, BoardgameQA, Natural Plan, and Moderator, respectively. "Indep.-Role" denotes an independent role ensemble baseline that uses the same instantiated agents and role profiles as AGENDAFLOW, but each agent solves the task independently without observing other agents or participating in phase-level discussion; "Generic" denotes the protocol-free multi-agent variant; "– Role", "– Routing", and "– Mod." denote removing functional role design, task-adaptive protocol routing, and moderator control from AGENDAFLOW. Standard deviations are reported in Appendix C.

functional role design and uses generic participant descriptions. w/o Protocol Routing disables task-adaptive protocol selection and applies a shared protocol across tasks. w/o Moderator removes moderator-governed phase control. We conduct the ablation study on four representative benchmarks using stratified samples of 500 examples from each benchmark.

As shown in Table 2, all variants underperform the full AGENDAFLOW framework. Generic-MA reduces the average accuracy from 62.01% to 56.14%, showing that the gains are not explained by multi-agent interaction alone. Indep.-Role reaches 58.90%, outperforming Generic-MA but remaining 3.11 points below AGENDAFLOW. This suggests that role-specialized independent reasoning and final aggregation are useful, but do not replace phase-by-phase deliberation and intermediate artifact sharing. Among the component ablations, removing the moderator causes the largest degradation, reducing the average accuracy to 56.64%. Disabling task-adaptive protocol routing also leads to a consistent drop, while removing role design results in a smaller decrease to 60.96%, suggesting that functional role specification contributes to performance but is less decisive than protocol routing and moderator control.

4.7 Process-Level Analysis

Since AGENDAFLOW is designed to organize multi-agent LLM reasoning through task-adaptive protocols, roles, and moderator-controlled phases, we further analyze intermediate discussion traces. We sample 400 instances and evaluate seven process-level metrics: constraint utilization, self-error detection, repair success, premature consensus, answer revision utility, process discipline, and perspective coverage. Higher values indicate better performance for all metrics except premature consensus. We use an LLM judge with fixed annotation rubrics for scalable evaluation. To assess annotation reliability, we manually inspect 20% of the judged samples. The annotations show reasonable agreement with human labels, with an average agreement of 83.2%, Cohen's κ of 0.785, and F1 of 0.825.

Table 3 reports the process-level results. AGENDAFLOW achieves the highest constraint utilization, self-error detection, repair success, answer revision utility, and process discipline. It also obtains the lowest premature consensus rate. These results indicate that structured reasoning improves not only final task accuracy, but also the quality of intermediate reasoning.

The improvement is especially clear on process discipline. Compared with generic multi-agent interaction, AGENDAFLOW produces more phase-consistent traces and fewer drifting discussions. This is consistent with the role of the moderator, which enforces phase boundaries, checks completion conditions, and prevents agents from moving to final agreement before required evidence or constraints have been examined. We also observe that removing the moderator slightly increases perspective coverage, but lowers final accuracy and process discipline. This suggests that more perspectives alone are not sufficient for effective multi-agent LLM reasoning. Without phase control, additional viewpoints may remain weakly integrated or disconnected from the final answer. AGENDAFLOW instead balances perspective diversity with structured convergence, which explains why it improves both reasoning-process quality and task performance.

4.8 Generalization and Compute-Controlled Analysis

Backbone generalization. We first examine whether the effectiveness of AGENDAFLOW depends on a particular backbone model. As shown in Table 4, AGENDAFLOW consistently achieves the highest average score across all five backbone models. Averaged over backbones, it obtains 60.97%,

Method	Constraint Utilization ↑	Self-Error Detection ↑	Repair Success ↑	Premature Consensus ↓	Revision Utility ↑	Process Discipline ↑	Perspective Coverage ↑
MAD	45.9	31.6	43.9	43.9	4.2	40.5	45.9
Generic-MA	50.7	35.9	48.4	46.9	5.8	49.0	52.3
w/o Role	60.3	48.1	58.1	40.7	10.3	65.9	59.7
w/o Routing	58.7	46.6	56.4	42.5	9.4	61.8	63.5
w/o Moderator	61.1	45.2	54.5	49.4	8.4	60.3	**71.4**
AGENDAFLOW	**68.5**	**54.9**	**63.7**	**36.6**	**13.2**	**71.9**	70.8

Table 3: Process-level analysis over 400 instances. Values are raw scores in percentage. Higher values are better for all metrics except premature consensus, where lower is better. Best values are shown in bold.

Backbone	SC	AutoGen	MAgICoRe	AGENDAFLOW
Kimi2	49.10	50.56	56.40	**57.57**
GPT-5.5	57.27	57.20	61.55	**62.73**
DeepSeek	53.33	53.73	58.67	**60.00**
Qwen3.6	57.33	56.93	61.25	**62.97**
GLM5.1	55.73	56.03	59.98	**61.57**
Average	54.55	54.89	59.57	**60.97**

Table 4: Backbone generalization results. Scores are average accuracy over Natural Plan, ACPBench Hard, and BoardgameQA. SC denotes Self-Consistency; DeepSeek denotes DeepSeek-V4-Pro.

Method	Tokens	ACP-H	BGQA	NP	Avg.
Self-Consistency	18.1K	37.6	76.0	36.7	50.1
AutoGen	18.0K	44.4	69.9	43.5	52.6
MAgICoRe	18.2K	46.8	76.1	46.9	56.6
AGENDAFLOW	18.3K	**49.3**	**76.5**	**48.8**	**58.2**

Table 5: Token-matched comparison under the same average token budget as AGENDAFLOW. Accuracy values are mean scores over five independent runs and are reported in percentage. Standard deviations are reported in Appendix C.

outperforming the strongest non-AGENDAFLOW baseline, MAgICoRe, by 1.40 absolute points. These results suggest that the benefit of protocol-guided interaction is not tied to a single model family. Detailed benchmark-level results with standard deviations are reported in Appendix 17.

Token-matched comparison. Since AGENDAFLOW introduces additional protocol selection, role-conditioned interaction, and moderator-controlled phase checking, we further examine whether its gains can be explained by increased test-time token usage. We conduct a token-matched comparison using the average token usage of AGENDAFLOW as the shared budget. The baselines are scaled within the same budget by increasing the number of self-consistency samples, debate rounds, or interaction turns. Total token usage is measured as the sum of prompt and completion tokens over all model calls.

As shown in Table 5, compute-scaled baselines improve under the enlarged budget, but AGENDAFLOW remains the best-performing method on all three evaluated benchmarks. Its average margin over the strongest token-matched baseline is 1.6 points. This result indicates that the improvement is not solely attributable to using more test-time tokens. Instead, the structured allocation of discussion phases, functional roles, and moderator control enables more effective use of the available computation.

5 Conclusion

We presented AGENDAFLOW, a task-adaptive protocol framework for structured multi-agent LLM reasoning. The framework selects task-specific deliberation protocols, instantiates functional roles, and uses a moderator to enforce phase boundaries and completion checks. Experiments across reasoning, planning, and rule- or evidence-based benchmarks show consistent improvements in task performance. Future work may extend this framework toward learned protocol policies and lower-cost deployment in interactive settings.

Limitations

AGENDAFLOW requires multi-call execution for protocol routing, role-conditioned deliberation, and moderator-based phase checking, which increases latency and complexity compared with simpler prompting or less structured multi-agent baselines. Although the token-matched analysis indicates that the observed gains are not merely a consequence of a larger token budget, reducing execution overhead remains important for practical use.

The current protocol library and role-slot schemas are manually specified. This design improves transparency and controllability, but may limit coverage for tasks whose deliberation structure falls outside the predefined protocol families. Future work should investigate data-driven protocol selection, role allocation, and phase-transition policies, and evaluate them in more open-ended, tool-augmented, and real-world decision-making environments.

References

Joseph A. Allen, Nale Lehmann-Willenbrock, and Steven G. Rogelberg, editors. 2015. *The Cambridge Handbook of Meeting Science*. Cambridge University Press.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2023. ChatEval: Towards better LLM-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*.

Huaben Chen, Wenkang Zhang, Yifei Liu, Qianying Liao, Gelei Liu, Yang Zhang, Zhaoxin Hong, and Yifei Chi. 2023a. Multi-agent consensus seeking via large language models. *arXiv preprint arXiv:2310.20151*.

Justin Chen, Archiki Prasad, Swarnadeep Saha, Elias Stengel-Eskin, and Mohit Bansal. 2025. MAgICoRe: Multi-agent, iterative, coarse-to-fine refinement for reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. 2024. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. *arXiv preprint arXiv:2309.13007*.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023b. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Carsten K. W. De Dreu and Michael A. West. 2001. Minority dissent and team innovation: The importance of participation in decision making. *Journal of Applied Psychology*, 86(6):1191–1201.

Yilun Du, Shuang Li, and Antonio Torralba. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*.

Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.

Randy Y. Hirokawa. 1985. Discussion procedures and decision-making performance. *Human Communication Research*, 12(2):203–224.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. *arXiv preprint arXiv:2306.02561*.

Simone Kauffeld and Nale Lehmann-Willenbrock. 2012. Meetings matter: Effects of team meetings on team and organizational success. *Small Group Research*, 43(2):130–158.

Mehran Kazemi, Quan Yuan, Deepti Bhatia, Najoung Kim, Xin Xu, Vaiva Imbrasaite, and Deepak Ramachandran. 2023. BoardgameQA: A dataset for natural language reasoning with contradictory information. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35:22199–22213.

Harsha Kokel, Michael Katz, Kavitha Srinivas, and Shirin Sohrabi. 2025. ACPBench: Reasoning about action, change, and planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Harsha Kokel, Michael Katz, Kavitha Srinivas, and Shirin Sohrabi. 2026. ACPBench Hard: Unrestrained reasoning about action, change, and planning. In *International Conference on Learning Representations*. Poster.

Desmond J. Leach, Steven G. Rogelberg, Peter B. Warr, and Jennifer L. Burnfield. 2009. Perceived meeting effectiveness: The role of design characteristics. *Journal of Business and Psychology*, 24(1):65–76.

Nale Lehmann-Willenbrock, Joseph A. Allen, and Simone Kauffeld. 2013. A sequential analysis of procedural meeting communication: How teams facilitate their meetings. *Journal of Applied Communication Research*, 41(4):365–388.

Guohao Li, Hasan Abed Al Kader Hammoud, and Hani Itani. 2023a. CAMEL: Communicative agents for "mind" exploration of large language model society. *arXiv preprint arXiv:2303.17760*.

Huao Li, Yu Quan Chong, Simon Stepputtis, Joseph Campbell, Dana Hughes, Michael Lewis, and Katia Sycara. 2023b. Theory of mind for multi-agent collaboration via large language models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 180–192.

Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024a. More agents is all you need. *arXiv preprint arXiv:2402.05120*.

Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, and Eugene Ie. 2024b. Improving multi-agent debate with sparse communication topology. *arXiv preprint arXiv:2406.11776*.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2023. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*.

Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2024. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *Proceedings of the First Conference on Language Modeling*.

Aman Madaan, Niket Tandon, and Prakhar Gupta. 2023. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*.

Jessica R. Mesmer-Magnus and Leslie A. DeChurch. 2009. Information sharing and team performance: A meta-analysis. *Journal of Applied Psychology*, 94(2):535–546.

Joseph E. Mroz, Joseph A. Allen, Debra C. Verhoeven, and Marissa L. Shuffler. 2018. Do we really need another meeting? the science of workplace meetings. *Current Directions in Psychological Science*, 27(6):484–491.

Carol T. Nixon and Glenn E. Littlepage. 1992. Impact of meeting procedures on meeting effectiveness. *Journal of Business and Psychology*, 6(3):361–369.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*.

Gerardo A. Okhuysen and Kathleen M. Eisenhardt. 2002. Integrating knowledge in groups: How formal interventions enable flexibility. *Organization Science*, 13(4):370–386.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.

Marshall Scott Poole and Jonelle Roth. 1989. Decision development in small groups IV: A typology of group decision paths. *Human Communication Research*, 15(3):323–356.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*.

Steven G. Rogelberg, Desmond J. Leach, Peter B. Warr, and Jennifer L. Burnfield. 2006. Not another meeting!: Are meeting time demands related to employee well-being? *Journal of Applied Psychology*, 91(1):83–96.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.

Helen B. Schwartzman. 1989. *The Meeting: Gatherings in Organizations and Communities*. Plenum Press, New York.

Noah Shinn, Federico Cassano, and Edward Berman. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*.

Zayne Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. 2024. MuSR: Testing the limits of chain-of-thought with multistep soft reasoning. In *International Conference on Learning Representations*. Spotlight.

Garold Stasser and William Titus. 1985. Pooling of unshared information in group decision making: Biased information sampling during discussion. *Journal of Personality and Social Psychology*, 48(6):1467–1478.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 2609–2634.

Xuezhi Wang, Jason Wei, and Dale Schuurmans. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Jason Wei, Xuezhi Wang, and Dale Schuurmans. 2022. Chain-of-thought prompting elicits reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*.

Gwen M. Wittenbaum, Andrea B. Hollingshead, and Isabel C. Botero. 2004. From cooperative to motivated information sharing in groups: Moving beyond the hidden profile paradigm. *Communication Monographs*, 71(3):286–310.

Anita Williams Woolley, Christopher F. Chabris, Alex Pentland, Nada Hashmi, and Thomas W. Malone. 2010. Evidence for a collective intelligence factor in the performance of human groups. *Science*, 330(6004):686–688.

Qingyun Wu, Gagan Bansal, and Jieyu Zhang. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junning Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. 2023. OpenAgents: An open platform for language agents in the wild. *arXiv preprint arXiv:2310.10634*.

Zhenran Xu, Senbao Shi, Baotian Hu, Jindi Yu, Dongfang Li, Min Zhang, and Yuxiang Wu. 2023. Towards reasoning in large language models via multi-agent peer review collaboration. *arXiv preprint arXiv:2311.08152*.

Shunyu Yao, Dian Yu, and Jeffrey Zhao. 2023. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*.

Shunyu Yao, Jeffrey Zhao, and Dian Yu. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Huaixiu Steven Zheng, Swaroop Mishra, Hugh Zhang, Xinyun Chen, Minmin Chen, Azade Nova, Le Hou, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, and Denny Zhou. 2024. NATURAL PLAN: Benchmarking LLMs on natural language planning. *arXiv preprint arXiv:2406.04520*.

Denny Zhou, Nathanael Schärli, and Le Hou. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.

Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. 2023. SOTOPIA: Interactive evaluation for social intelligence in language agents. *arXiv preprint arXiv:2310.11667*.

A Additional Method Details

A.1 Protocol Selection Taxonomy

AGENDAFLOW selects a primary meeting protocol according to the dominant objective of the input task. The taxonomy considers three factors: the expected final output, the main reasoning object, and the process risk that the deliberation should control. Table 6 summarizes the protocol families used in AGENDAFLOW.

When multiple protocol signals are present, AGENDAFLOW selects the protocol that matches the required final output. Secondary reasoning needs are handled by activating optional roles within the selected protocol.

A.2 Complete Protocol Library

AGENDAFLOW represents each task-adaptive meeting protocol as a structured phase template consisting of a phase objective, candidate roles for activation, an intermediate artifact, and a completion criterion. After a protocol is selected, the role-slot set is instantiated once and remains fixed throughout the deliberation. Across phases, the underlying role composition remains fixed, while the moderator dynamically activates a subset of roles according to the current phase objective, accumulated artifacts, and unresolved issues.

Each protocol contains four phases. The four-phase structure is not a shared generic template: each protocol defines its own phase sequence according to the task objective and the dominant deliberation risk. The protocol designs follow meeting-science-inspired principles such as agenda sequencing, role specialization, structured information sharing, explicit dissent, and phase-level closure.

A.2.1 Planning Protocol

The Planning Protocol is used when the task requires constructing an executable plan, schedule, route, or action sequence under constraints. As shown in Table 7, its phase structure emphasizes goal clarification, constraint externalization, plan construction, and execution checking.

A.2.2 Problem-Solving Protocol

The Problem-Solving Protocol is used for failures, anomalies, inconsistencies, blocked states, or invalid outcomes. Table 8 follows the same logic as the main text: problem framing, diagnostic inquiry, intervention design, and answer convergence.

A.2.3 Evidence-and-Rule Verification Protocol

The Evidence-and-Rule Verification Protocol is used when the task requires deriving a conclusion from rules, states, narrative facts, clues, or dispersed evidence. Table 9 summarizes its evidence-centered phase structure.

A.2.4 Decision-Selection Protocol

The Decision-Selection Protocol is used when the task requires choosing among candidate options, strategies, paths, or allocation plans. As shown in Table 10, the protocol first fixes the decision boundary, then anchors the evaluation criteria, compares candidates under a shared standard, and finally closes the deliberation through a conditional decision. This structure is designed to reduce criteria drift, option drift, and premature consensus in multi-agent decision making.

A.3 Role-Slot Schemas

Table 11 reports the protocol-specific role-slot schemas used by AGENDAFLOW. Each protocol contains three core role slots and seven optional role slots. Core slots define the mandatory functional coverage required by the selected protocol, while optional slots provide additional perspectives that can be selected according to the task context and target participant count. The slots are internal functional identifiers rather than fixed participant identities; concrete participant titles, backgrounds, thinking styles, and unique contributions are instantiated from these slots for each input task.

A.4 Moderator-Governed Phase Execution

After a protocol and its role-slot schema are instantiated, AGENDAFLOW executes the deliberation through a moderator-governed phase procedure. The moderator is not a domain participant and does not independently introduce new task-specific evidence. Instead, it maintains the deliberation state, activates the relevant roles for each phase, synthesizes intermediate artifacts, and checks whether the current phase has satisfied its completion criterion.

Moderator state. For each phase ϕ_t, the moderator maintains a state

$$h_t = (x, \phi_t, H_{<t}, A_{<t}, U_t),$$

where x is the input task, ϕ_t is the current phase, $H_{<t}$ denotes the previous deliberation history, $A_{<t}$ denotes the artifacts produced by previous phases,

Protocol family	Selected when the task primarily asks the system to	Main reasoning object	Dominant process risk	Protocol-level response
Planning	Construct a feasible plan, schedule, route, action sequence, or arrangement under constraints.	Goals, constraints, resources, time slots, locations, actions, preconditions, and effects.	Missing hard constraints, producing an infeasible plan, or failing to verify intermediate states.	Clarify goals; enumerate constraints; construct a candidate plan; verify feasibility; produce the final plan.
Problem-Solving	Diagnose a current failure, anomaly, inconsistency, invalid outcome, or blocked situation and identify corrective actions.	Symptoms, causes, evidence, failure conditions, and intervention options.	Jumping from symptoms to solutions without identifying the underlying cause.	Frame the problem; reconstruct facts; test causal hypotheses; map fixes to causes; converge on corrective actions.
Evidence-and-Rule Verification	Derive a conclusion from rules, states, narrative facts, clues, or dispersed evidence.	Evidence spans, rules, state transitions, claims, contradictions, supporting evidence, and counter-evidence.	Reaching unsupported conclusions, overlooking counter-evidence, or misapplying rules.	Identify the target claim or rule; collect relevant evidence; check consistency and counter-evidence; synthesize a grounded conclusion.
Decision-Selection	Choose the best, valid, or most appropriate option among explicit candidates under criteria or constraints.	Candidate options, decision criteria, trade-offs, hard constraints, and preferences.	Criteria drift, over-weighting soft preferences, or selecting an option that violates hard constraints.	Define criteria; compare options; eliminate invalid choices; analyze trade-offs; select the final option with rationale.

Table 6: Protocol-selection taxonomy used by AGENDAFLOW.

and U_t denotes the set of unresolved issues identified so far. Compared with the state definition in the main text, we explicitly include $A_{<t}$ to distinguish raw dialogue history from structured phase artifacts.

Phase execution. At the beginning of each phase, the moderator constructs a phase instruction from the selected task-adaptive protocol template, the current phase objective, the activated role set, previous artifacts, and unresolved issues. Only roles relevant to the current phase are asked to contribute. Each activated agent produces a role-conditioned response, after which the moderator synthesizes the responses into a phase artifact. The artifact is then checked against the completion criterion specified by the protocol. The full phase-execution procedure is summarized in Algorithm 1.

B Prompt Templates

B.1 Prompt Template Overview

We implement AGENDAFLOW with a small collection of structured prompt templates. Each template defines one control operation in the deliberation process, such as selecting a protocol, instantiating agents, moderating a phase, synthesizing intermediate artifacts, checking completion, or producing the final answer. To make these prompts inspectable, we present them as prompt cards. Each prompt card specifies five components: the template's purpose, its required inputs, the main instruction given to the model, any execution constraints, and the expected output schema. Task-specific information is introduced only through placeholders such as `[TASK]`, `[PROTOCOL]`, `[PHASE]`, `[ROLE_PROFILE]`, `[AGENT_RESPONSES]`, and `[ARTIFACTS]`. Figures 4–10 provides the complete prompt cards for task routing, role instantiation, phase moderation, agent response generation, artifact synthesis, completion checking, and final answer generation.

B.2 Task Routing Prompt

The task routing prompt maps each input instance to one primary task-adaptive meeting protocol before deliberation begins. The routing decision is based on the protocol-selection taxonomy in Section A.1. In particular, the model is instructed to consider the required final output, the main reasoning object, and the dominant process risk. When multiple protocol signals are present, the selected protocol is the one that best matches the required final output, while secondary reasoning needs are handled through optional role activation.

Phase	Objective	Candidate roles for activation	Phase artifact	Completion criterion
Goal Anchoring	Clarify the target outcome, output form, and success conditions.	Goal Clarifier; Task Interpreter	Goal specification	The goal and required output are explicit.
Constraint Mapping	Identify hard constraints, soft preferences, resource limits, and ordering relations.	Constraint Auditor; State Tracker	Constraint map	Explicit constraints are covered and implicit constraints are marked.
Plan Assembly	Construct a candidate plan that satisfies the stated goal and constraints.	Plan Composer; Feasibility Analyst	Candidate plan	The plan covers the required steps and respects major constraints.
Execution Check	Verify feasibility, detect conflicts, and identify residual risks.	Feasibility Verifier; Risk Checker	Verified plan	No unresolved hard-constraint violation remains.

Table 7: Phase structure of the Planning Protocol.

Phase	Objective	Candidate roles for activation	Phase artifact	Completion criterion
Problem Framing	Distinguish the core problem from symptoms, impacts, and adjacent issues.	Problem Framer; Scope Analyst	Problem boundary	The problem, symptoms, and scope are clearly separated.
Diagnostic Inquiry	Reconstruct relevant facts, organize the timeline, and compare causal hypotheses against supporting and counter evidence.	Evidence Inspector; Causal Analyst; Skeptic	Diagnostic summary	Major causal hypotheses have been checked against evidence.
Intervention Design	Map interventions to diagnosed causes and distinguish containment, corrective, and preventive actions.	Intervention Designer; Feasibility Verifier	Cause–action map	Proposed actions address the diagnosed causes rather than only symptoms.
Answer Convergence	Consolidate the diagnosis, interventions, residual uncertainty, and final response.	Solution Synthesizer; Uncertainty Checker	Final response	The response satisfies the original task objective and states necessary uncertainty.

Table 8: Phase structure of the Problem-Solving Protocol.

B.3 Role Instantiation Prompt

After protocol selection, AGENDAFLOW instantiates a fixed set of agents from the protocol-specific role-slot schema in Section A.3. This step converts abstract functional slots into task-adapted agent profiles while preserving the deliberation function assigned by the selected protocol, which takes the input task, selected protocol, role-slot schema, and target agent count as runtime inputs, and returns the agent profiles used in all subsequent phases.

The instantiated roles are fixed for the remainder of the deliberation. Phase-level moderation may activate different subsets of these roles across phases, but it does not regenerate or rewrite the role profiles.

B.4 Phase Moderation Prompt

At each protocol phase, the moderator determines how the instantiated agents should participate in the current step of deliberation. The phase moderation prompt receives the selected protocol, current phase objective, available role profiles, previous artifacts, and unresolved issues, and produces a phase-level execution instruction. It does not revise the protocol assignment or regenerate agent roles.

The moderation output is used to control role participation and phase-local objectives. It may restrict which agents speak, specify the order of contributions, and identify unresolved issues to track, but it does not directly produce the final answer.

Phase	Objective	Candidate roles for activation	Phase artifact	Completion criterion
Inference Targeting	Identify the question, claim, rule object, or state object to be verified.	Task Interpreter; Claim Analyst	Inference target	The target inference and output form are explicit.
Evidence Retrieval	Collect relevant facts, rules, clues, and state information.	Evidence Inspector; Rule Interpreter	Evidence–rule set	Key supporting materials are identified.
Consistency Testing	Check conflicts, ambiguity, and counter-evidence across rules, states, and facts.	State Tracker; Contradiction Challenger	Consistency report	Major ambiguities and counter-evidence are identified.
Grounded Judgment	Produce a conclusion grounded in the verified evidence or rules.	Evidence Synthesizer; Verification Analyst	Grounded conclusion	The conclusion is supported by the evidence or rule structure.

Table 9: Phase structure of the Evidence-and-Rule Verification Protocol.

Phase	Objective	Candidate roles for activation	Phase artifact	Completion criterion
Decision Boundary Locking	Clarify the decision question, review scope, and candidate option set.	Decision Goal Guardian; Option Comparison Analyst; Execution Feasibility Analyst	Decision scope; option set	The decision object is clear and the options are comparable.
Criteria Anchoring	Fix evaluation criteria and distinguish hard constraints, soft preferences, and weighting assumptions.	Decision Goal Guardian; Risk–Constraint Analyst; Cost–Benefit Analyst	Criteria set; hard constraints	Evaluation criteria are stable and constraints are separated from preferences.
Comparative Challenge	Compare options under the same criteria and expose objections, evidence gaps, and fragile assumptions.	Option Comparison Analyst; Counter-Challenge Analyst; Long-Term Impact Analyst	Comparison matrix; objections	Each option has been compared under the same criteria and major objections are recorded.
Conditional Closure	Make the final selection and specify conditions, execution implications, and reopening triggers.	Trade-off Analyst; Risk–Constraint Analyst; Execution Feasibility Analyst	Decision statement; decision conditions	The selected option satisfies hard constraints and residual objections are conditionally addressed.

Table 10: Phase structure of the Decision-Selection Protocol.

B.5 Agent Response Generation Prompt

During each protocol phase, active agents produce role-conditioned contributions according to the moderator instruction. The agent response prompt conditions each response on the fixed role profile, current phase objective, prior artifacts, and unresolved issues. This step generates phase-local analysis rather than a final answer.

Agent responses are used as inputs to the subsequent artifact synthesis step. They are required to follow the assigned role responsibility and remain grounded in the task, existing artifacts, and phase-local moderation instruction.

B.6 Artifact Synthesis Prompt

After active agents complete a phase, AGENDAFLOW synthesizes their contributions into a phase artifact. The artifact synthesis prompt takes the agent responses, current phase objective, previous artifacts, and unresolved issues as inputs, and produces a compact intermediate representation used by later phases.

The synthesized artifact is the only phase output carried forward by default. It preserves conclusions, supporting evidence, constraints, and open issues, while filtering redundant or unsupported agent statements.

Protocol	Core role slots	Optional role slots
Planning	Goal–constraint perspective; Plan-construction perspective; Feasibility-verification perspective	State-tracking perspective; Resource-capacity perspective; Temporal-ordering perspective; Alternative-plan perspective; Risk-contingency perspective; Execution-scene perspective; Validation-closure perspective
Problem-Solving	Problem-surface perspective; System-causality perspective; Intervention-design perspective	Timeline-reconstruction perspective; Process-mechanism perspective; Data-evidence perspective; Execution-scene perspective; Object-impact perspective; Resource-constraint perspective; Validation-closure perspective
Evidence-and-Rule Verification	Inference-target perspective; Evidence-inspection perspective; Rule–state verification perspective	Contradiction-challenge perspective; Ambiguity-resolution perspective; Counter-evidence perspective; State-transition perspective; Evidence-sufficiency perspective; Grounded-synthesis perspective; Assumption-boundary perspective
Decision-Selection	Decision-boundary perspective; Criteria-constraint perspective; Comparative-tradeoff perspective	Decision-holder perspective; Execution-feasibility perspective; Risk-compliance perspective; Cost-utility perspective; Stakeholder-voice perspective; Long-term-strategy perspective; Independent-challenge perspective

Table 11: Protocol-specific role-slot schemas used by AGENDAFLOW. Each protocol contains three core role slots and seven optional role slots. Core slots are mandatory for the selected protocol, while optional slots are selected according to the input task and target participant count.

B.7 Completion Checking Prompt

After each phase artifact is synthesized, AGENDAFLOW checks whether the artifact satisfies the completion criterion of the current protocol phase. The completion checking prompt takes the phase objective, synthesized artifact, and unresolved issues as inputs, and returns a decision on whether the system should advance or repeat the phase.

The checker is used only for phase control. It does not revise the artifact, introduce new reasoning, or produce the final answer; incomplete phases are returned to moderation with the missing items explicitly recorded.

B.8 Final Answer Generation Prompt

After all protocol phases are completed, AGENDAFLOW generates the final response from the accumulated phase artifacts. The final answer prompt receives the original task, completed artifacts, unresolved issues, and required answer format as inputs, and produces the output submitted for evaluation.

This step does not reopen deliberation or introduce new evidence. It uses the synthesized artifacts as the basis for the final response and follows the answer format specified by the task or benchmark.

C Complete Experimental Results

C.1 Complete Main Results

Table 12 reports the complete benchmark-level main results with standard deviations across five independent runs. To improve readability, we separate benchmark-level averages from subtask-level scores. Specifically, Table 12 includes the three standalone benchmarks, the averaged MuSR and Natural Plan results, and the final average over the five benchmark-level scores, while Table 13 provides the corresponding subtask-level results for MuSR and Natural Plan. Together, these two tables provide the complete version of the main results summarized in Table 1.

C.2 Statistical Reliability

To assess whether the gains of AGENDAFLOW are reliable across test instances, we compare it with the strongest non-AGENDAFLOW baseline on each benchmark using paired tests. For each instance, we average correctness over five random seeds and compute the paired difference between AGENDAFLOW and the baseline. We estimate 95% confidence intervals using 10,000 paired bootstrap resamples over instances, and compute two-sided p-values using paired randomization tests with 100,000 sign-flip samples. Holm-Bonferroni

Algorithm 1 AGENDAFLOW Phase Execution

Require: Input task x, protocol library $\mathcal{P}$, role-slot schemas $\mathcal{S}$, maximum phase repeat M
Ensure: Final answer y

Infer task type τ from x
Select protocol $P_\tau = \{(\phi_t, o_t, a_t, R_t, c_t)\}_{t=1}^{T}$
Instantiate role set R_τ from schema S_τ
Initialize history $H \leftarrow \emptyset$, artifacts $A \leftarrow \emptyset$, unresolved issues $U \leftarrow \emptyset$
for $t = 1$ to T **do**
  $m \leftarrow 0$
  repeat
    Activate roles R_t according to phase ϕ_t and unresolved issues U
    Moderator issues phase instruction based on (x, ϕ_t, H, A, U)
    Collect responses from activated roles
    Moderator synthesizes responses into artifact $\hat{a}_t$
    Update unresolved issues U
    Check whether $\hat{a}_t$ satisfies completion criterion c_t
    $m \leftarrow m + 1$
  until c_t is satisfied or $m = M$ or U stops changing
  Append $\hat{a}_t$ to A and update H
end for
Generate final answer y from x, protocol artifacts A, and remaining unresolved issues U
return y

correction is applied across the five benchmark-level comparisons.

As shown in Table 14, AGENDAFLOW significantly outperforms the strongest non-AGENDAFLOW baseline on all five benchmark-level comparisons after Holm-Bonferroni correction. The confidence intervals are positive across all benchmarks, indicating that the observed gains are not driven by a small number of unpaired instances.

C.3 Complete Difficulty-Stratified Results

Table 15 reports the complete numerical results underlying the complexity-stratified analysis in Figure 3. Following Section 4.5, we assign each instance a normalized complexity score from 0 to 10 and divide examples in each benchmark into five equal-width complexity-score bins. The table reports the accuracy of each method within each bin. These results provide the complete table-level evidence behind the trend visualized in Figure 3: the advantage of AGENDAFLOW becomes larger as instance complexity increases.

C.4 Complete Ablation Results

Table 16 reports the complete ablation results corresponding to Table 2, with standard deviations across five independent runs. Overall, the results remain consistent with the main-text analysis: all variants underperform the full AGENDAFLOW framework, with the largest drops observed for the generic multi-agent variant and the variant without moderator-governed phase control.

C.5 Complete Generalization Results

Table 17 reports the complete generalization results.

Across five backbone models, AGENDAFLOW consistently achieves the best average score. Compared with the strongest non-AGENDAFLOW baseline, MAGICORE, AGENDAFLOW improves the average score from 59.57% to 60.97%, yielding a 1.4-point average gain. The gains are most consistent on Natural Plan and ACPBench Hard, suggesting that task-adaptive protocol control generalizes across model families particularly well for planning and action-state reasoning tasks.

C.6 Complete Token-Matched Results

We further provide the complete token-matched results with standard deviations in Table 18. Under the same average token budget as AGENDAFLOW, MAGICORE becomes the strongest non-AGENDAFLOW baseline, achieving an average score of 56.6%. Nevertheless, AGENDAFLOW still obtains the best average score of 58.2%, improving

Task Routing Prompt

Purpose. Select one primary task-adaptive meeting protocol for the input task before deliberation begins.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL_TAXONOMY]`: definitions and selection criteria for the available protocols.

Instruction. Given the input task, select exactly one protocol from: Planning, Problem-Solving, Evidence-and-Rule Verification, and Decision-Selection. Base the decision primarily on the final output required by the task. Use the reasoning object and dominant process risk only when the final-output signal is ambiguous.

Protocol selection criteria.

- Planning: the task asks for a feasible plan, schedule, route, action sequence, arrangement, or allocation under constraints.
- Problem-Solving: the task asks to diagnose a failure, anomaly, inconsistency, invalid outcome, or blocked situation and identify corrective actions.
- Evidence-and-Rule Verification: the task asks to derive a conclusion from rules, states, clues, narrative facts, or dispersed evidence.
- Decision-Selection: the task asks to choose the best, valid, or most appropriate option among explicit candidates under criteria or constraints.

Constraints.

- Select exactly one primary protocol.
- Do not answer the task.
- Do not generate a plan, diagnosis, proof, or final decision.
- If multiple protocols appear relevant, choose the one aligned with the required final answer format.
- Record secondary reasoning needs only as auxiliary signals for later role activation.

Output schema.

```
Protocol: [Planning / Problem-Solving /
          Evidence-and-Rule Verification /
          Decision-Selection]

Justification: [one concise sentence explaining the routing decision]

Secondary needs: [optional list of secondary reasoning needs,
                or "None"]
```

Figure 4: Prompt card for the task routing template.

over MAGICoRE by 1.6 absolute points. This result suggests that the gains of AgendaFlow are not merely due to increased token usage, but also come from a more effective allocation of test-time computation through task-adaptive protocols, functional roles, and moderator-guided phase control.

Role Instantiation Prompt

Purpose. Instantiate a fixed set of task-adapted agents from the selected protocol's functional role slots.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL]`: the protocol selected by the routing step.
- `[ROLE_SLOT_SCHEMA]`: the core and optional role slots associated with the selected protocol.
- `[TARGET_AGENT_COUNT]`: the number of agents to instantiate.

Instruction. Generate one agent profile for each selected role slot. Preserve the functional meaning of the slot, but adapt the agent title, background, and reasoning style to the input task. Prioritize core role slots required by the selected protocol. If optional roles are needed to reach the target agent count, select the optional slot that best addresses the dominant task-specific risk.

Role selection rule.

- Include all core role slots for the selected protocol.
- Add optional role slots until `[TARGET_AGENT_COUNT]` is reached.
- Prefer optional slots that improve constraint checking, evidence coverage, state tracking, or objection handling.
- Do not create roles outside the provided role-slot schema.

Constraints.

- Instantiate exactly `[TARGET_AGENT_COUNT]` agents.
- Keep each agent tied to one original role slot.
- Do not solve the task.
- Do not assign final-answer authority to any single agent.
- Avoid overlapping responsibilities across agents.

Output schema.

```
Agent 1:
  Slot: [original role slot]
  Title: [concise task-adapted participant title]
  Background: [task-relevant expertise or perspective]
  Thinking style: [reasoning behavior to follow]
  Primary responsibility: [deliberation function]

Agent 2:
  Slot: [...]
  Title: [...]
  Background: [...]
  Thinking style: [...]
  Primary responsibility: [...]

...
```

Figure 5: Prompt card for the role instantiation template. The template converts protocol-specific role slots into fixed agent profiles for later phase execution.

Method	ACPBench	ACP-Hard	BoardgameQA	MuSR Avg.	Natural Plan Avg.	Avg.
Direct	58.42 (±0.48)	35.08 (±0.61)	73.00 (±0.37)	60.26 (±0.53)	33.75 (±0.73)	52.10 (±0.36)
CoT	57.83 (±0.62)	36.50 (±0.76)	74.58 (±0.49)	73.64 (±0.61)	37.78 (±0.81)	56.07 (±0.43)
Self-Consistency	59.08 (±0.44)	36.83 (±0.52)	75.58 (±0.35)	74.67 (±0.42)	34.92 (±0.56)	56.22 (±0.34)
MAD	67.33 (±0.93)	44.25 (±1.12)	61.58 (±1.35)	56.71 (±1.04)	45.34 (±1.09)	55.04 (±0.71)
AutoGen	65.50 (±1.06)	42.83 (±1.24)	68.25 (±1.18)	64.00 (±0.92)	40.58 (±0.98)	56.23 (±0.68)
ReConcile	68.42 (±0.74)	45.17 (±0.87)	<u>76.10</u> (±0.68)	74.68 (±0.60)	45.90 (±0.83)	62.05 (±0.52)
MAgICoRe	68.92 (±0.71)	45.75 (±0.84)	76.05 (±0.66)	<u>74.86</u> (±0.58)	<u>47.40</u> (±0.78)	<u>62.60</u> (±0.50)
DyLAN	<u>69.15</u> (±0.90)	<u>46.05</u> (±1.05)	75.40 (±0.82)	72.53 (±0.73)	46.63 (±0.89)	61.95 (±0.60)
AGENDAFLOW	**71.58** (±0.57)	**49.67** (±0.69)	**77.25** (±0.46)	**76.28** (±0.48)	**48.83** (±0.66)	**64.72** (±0.41)

Table 12: Benchmark-level main results with standard deviations. All values are accuracy scores in percentage, reported as the mean over five independent runs with different random seeds. Standard deviations are shown in parentheses. The final average is computed over the five benchmark-level scores. Best results are in bold and second-best results are underlined.

Phase Moderation Prompt

Purpose. Control the execution of the current protocol phase by selecting active roles, defining the local objective, and specifying the expected phase artifact.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL]`: the selected task-adaptive meeting protocol.
- `[PHASE]`: the current phase name and objective.
- `[ROLE_PROFILES]`: the fixed agent profiles instantiated for the task.
- `[ARTIFACTS]`: artifacts produced in previous phases.
- `[UNRESOLVED_ISSUES]`: open constraints, contradictions, missing evidence, or uncertain assumptions.

Instruction. Generate a phase-level moderation instruction for the current step. Select the agents whose roles are relevant to the phase objective, assign their contribution order if needed, and specify what artifact should be produced after their responses. Use previous artifacts and unresolved issues to focus the discussion.

Moderation rules.

- Activate only roles needed for the current phase objective.
- Preserve the selected protocol and fixed role profiles.
- Prioritize unresolved constraints, unsupported claims, conflicting evidence, and incomplete intermediate artifacts.
- Specify the expected artifact in a form that can be checked by the completion step.

Constraints.

- Do not solve the task directly.
- Do not generate the final answer.
- Do not introduce new agent roles.
- Do not change the selected protocol.
- Do not discard unresolved issues unless they are explicitly addressed by prior artifacts.

Output schema.

```
Phase: [current phase name]

Active agents: [ordered list of agent IDs or role names]

Moderator instruction:
  [phase-specific instruction for the active agents]

Expected artifact:
  [artifact to be produced after this phase]

Issues to track:
  [constraints, contradictions, missing evidence, or assumptions
   that must remain visible during this phase]
```

Figure 6: Prompt card for the phase moderation template.

Method	MuSR			Natural Plan		
	Murder	Object	Team	Calendar	Meeting	Trip
Direct	66.40 (±0.74)	50.78 (±0.81)	63.60 (±0.69)	34.23 (±0.92)	35.74 (±0.88)	31.27 (±1.14)
CoT	81.20 (±0.83)	61.72 (±0.95)	78.00 (±0.78)	49.85 (±1.05)	**50.75** (±1.12)	12.73 (±1.26)
Self-Consistency	82.30 (±0.58)	62.90 (±0.66)	78.81 (±0.54)	46.20 (±0.71)	47.20 (±0.78)	11.36 (±0.93)
MAD	58.80 (±1.48)	44.14 (±1.62)	67.20 (±1.39)	60.69 (±1.56)	42.94 (±1.31)	<u>32.40</u> (±1.72)
AutoGen	70.10 (±1.34)	54.10 (±1.46)	67.80 (±1.29)	54.80 (±1.43)	39.40 (±1.38)	27.54 (±1.65)
ReConcile	82.05 (±0.89)	63.05 (±0.97)	78.95 (±0.82)	58.20 (±1.18)	47.65 (±0.96)	31.85 (±1.43)
MAgICoRe	<u>82.35</u> (±0.86)	<u>63.18</u> (±0.94)	<u>79.05</u> (±0.79)	61.10 (±1.12)	<u>49.20</u> (±0.91)	31.90 (±1.38)
DyLAN	79.60 (±1.02)	61.05 (±1.15)	76.95 (±1.06)	<u>61.35</u> (±1.24)	46.80 (±1.10)	31.75 (±1.52)
AGENDAFLOW	**83.60** (±0.71)	**64.70** (±0.80)	**80.54** (±0.66)	**63.80** (±0.89)	49.10 (±0.95)	**33.59** (±1.08)

Table 13: Subtask-level main results with standard deviations for MuSR and Natural Plan. All values are accuracy scores in percentage, reported as the mean over five independent runs with different random seeds. Benchmark-level averages are reported in Table 12. Best results are in bold and second-best results are underlined.

Agent Response Generation Prompt

Purpose. Generate a role-conditioned contribution for the current protocol phase.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL]`: the selected task-adaptive meeting protocol.
- `[PHASE]`: the current phase name and objective.
- `[ROLE_PROFILE]`: the fixed profile of the active agent.
- `[MODERATOR_INSTRUCTION]`: the phase-level instruction assigned by the moderator.
- `[ARTIFACTS]`: artifacts produced in previous phases.
- `[UNRESOLVED_ISSUES]`: open constraints, contradictions, missing evidence, or uncertain assumptions.

Instruction. Respond only from the perspective of the assigned role. Use the moderator instruction to determine the scope of the contribution. Refer to prior artifacts when they are relevant, and explicitly address unresolved issues that fall under the role's responsibility.

Response rules.

- Preserve the reasoning function specified by `[ROLE_PROFILE]`.
- Focus on the current phase objective rather than the whole task.
- Distinguish supported conclusions from assumptions.
- Surface conflicts, missing constraints, or weak evidence when they affect the phase artifact.

Constraints.

- Do not produce the final answer.
- Do not change the selected protocol, phase objective, or role profile.
- Do not introduce facts not grounded in the task or prior artifacts.
- Do not resolve issues outside the assigned role unless they directly affect the current phase.

Output schema.

```
Role: [agent role name]

Phase contribution:
  [role-conditioned analysis, proposal, objection, or verification]

Relevant evidence or constraints:
  [task facts, rules, constraints, or artifacts used]

Issues identified:
  [new or remaining issues, or "None"]

Suggested artifact update:
  [concise update that should be considered by synthesis]
```

Figure 7: Prompt card for the agent response generation template.

Benchmark	Baseline	Base.	AF	Gain	95% CI	p_{Holm}
ACPBench	DyLAN	69.15	71.58	+2.43	[+1.28, +3.61]	$<$ .001
ACP-Hard	DyLAN	46.05	49.67	+3.62	[+2.05, +5.14]	$<$ .001
BoardgameQA	ReConcile	76.10	77.25	+1.15	[+0.24, +2.03]	.036
MuSR	MAgICoRe	74.86	76.28	+1.42	[+0.46, +2.35]	.016
Natural Plan	MAgICoRe	47.40	48.83	+1.43	[+0.39, +2.51]	.021

Table 14: Paired significance tests between AGENDAFLOW and the strongest non-AGENDAFLOW baseline on each benchmark. Base. and AF denote the mean accuracy of the baseline and AGENDAFLOW, respectively. Confidence intervals are estimated with paired bootstrap resampling over instances. Reported p-values are Holm-Bonferroni corrected across the five benchmark-level comparisons.

Artifact Synthesis Prompt

Purpose. Consolidate the active agents' phase contributions into a structured artifact for subsequent deliberation.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL]`: the selected task-adaptive meeting protocol.
- `[PHASE]`: the current phase name and objective.
- `[MODERATOR_INSTRUCTION]`: the phase-level instruction.
- `[AGENT_RESPONSES]`: role-conditioned contributions from active agents.
- `[PREVIOUS_ARTIFACTS]`: artifacts produced by earlier phases.
- `[UNRESOLVED_ISSUES]`: issues carried into the current phase.

Instruction. Synthesize the agent responses into a single phase artifact. Preserve claims that are supported by the task, prior artifacts, or agent-specific reasoning. Identify remaining disagreements, missing evidence, constraint violations, or assumptions that should be carried forward.

Synthesis rules.

- Align the artifact with the current phase objective.
- Merge compatible contributions instead of listing responses separately.
- Retain evidence, rules, constraints, or state updates needed by later phases.
- Mark unsupported, conflicting, or incomplete claims as unresolved.

Constraints.

- Do not generate the final answer unless this is the final-answer phase.
- Do not introduce facts not present in the task, prior artifacts, or agent responses.
- Do not discard unresolved issues without explicit support.
- Do not attribute authority to an agent solely because it spoke last.

Output schema.

```
Phase artifact:
  [consolidated intermediate result for the current phase]

Supported conclusions:
  [claims or updates supported by evidence, rules, or constraints]

Evidence and constraints used:
  [task facts, rules, constraints, or previous artifacts used]

Unresolved issues:
  [remaining conflicts, missing evidence, assumptions, or "None"]

Carry-forward notes:
  [information that subsequent phases must preserve]
```

Figure 8: Prompt card for the artifact synthesis template.

Completion Checking Prompt

Purpose. Determine whether the current phase artifact satisfies the completion criterion required to advance to the next protocol phase.

Inputs.

- `[TASK]`: the original input task.
- `[PROTOCOL]`: the selected task-adaptive meeting protocol.
- `[PHASE]`: the current phase name and objective.
- `[COMPLETION_CRITERION]`: the condition required to complete the phase.
- `[PHASE_ARTIFACT]`: the synthesized artifact produced for the current phase.
- `[UNRESOLVED_ISSUES]`: remaining constraints, contradictions, missing evidence, or assumptions.

Instruction. Evaluate whether the phase artifact is sufficient for the current phase objective. Check the artifact against the completion criterion and determine whether unresolved issues still block progress. Return an explicit advance-or-repeat decision with the missing items needed for completion.

Checking rules.

- Mark the phase complete only if the artifact satisfies the stated criterion.
- Treat unresolved hard constraints, contradictions, or missing evidence as blockers.
- Do not reward verbosity; evaluate sufficiency rather than length.
- Preserve unresolved issues that remain relevant to later phases.

Constraints.

- Do not rewrite the phase artifact.
- Do not generate new task reasoning.
- Do not produce the final answer.
- Do not change the protocol, phase objective, or completion criterion.
- Do not mark a phase complete when required evidence or constraints are absent.

Output schema.

```
Completion decision:
  [Advance / Repeat phase]

Criterion satisfied:
  [Yes / No]

Missing items:
  [constraints, evidence, checks, or artifact components
   still required; or "None"]

Blocking issues:
  [unresolved contradictions, unsupported assumptions,
   constraint violations, or "None"]

Instruction for next step:
  [advance to next phase, or repeat current phase with
   specified missing items]
```

Figure 9: Prompt card for the completion checking template.

Final Answer Generation Prompt

Purpose. Generate the final response from the completed protocol artifacts.

Inputs.

- [TASK]: the original input task.
- [PROTOCOL]: the selected task-adaptive meeting protocol.
- [COMPLETED_ARTIFACTS]: synthesized artifacts from completed phases.
- [UNRESOLVED_ISSUES]: remaining issues, if any.
- [ANSWER_FORMAT]: the required output format for the task or benchmark.

Instruction. Produce the final answer using the completed artifacts as the sole reasoning basis. Respect the required answer format. If unresolved issues remain, use the most supported conclusion and expose uncertainty only when the task format permits it.

Answer rules.

- Use conclusions, constraints, evidence, and state updates recorded in the completed artifacts.
- Follow the requested answer type, such as option label, entity, sequence, plan, or short textual response.
- Prefer concise output when the benchmark expects exact matching or option selection.
- Include explanation only when the task explicitly requires or permits it.

Constraints.

- Do not introduce new facts, constraints, or evidence.
- Do not reopen earlier phases or call additional agents.
- Do not mention internal roles, phases, or artifacts unless requested.
- Do not output multiple alternatives unless the task requires them.
- Do not include reasoning traces when only the final answer is requested.

Output schema.

```
Final answer:
  [answer in the required format]

Optional explanation:
  [only if permitted by the task or evaluation format]

Confidence note:
  [only if unresolved issues remain and uncertainty reporting is allowed]
```

Figure 10: Prompt card for the final answer generation template.

Benchmark	Complexity Bin	Direct	CoT	Self-Consistency	MAD	AutoGen	ReConcile	MAgICoRe	DyLAN	AgendaFlow
ACPBench	0–2	89.85	90.80	90.25	93.10	92.40	93.45	93.60	<u>93.75</u>	**94.35**
	2–4	76.10	75.80	77.00	83.50	81.70	84.20	84.55	<u>84.80</u>	**86.10**
	4–6	60.20	59.60	61.40	68.70	66.90	70.10	70.45	<u>70.70</u>	**73.00**
	6–8	43.50	42.60	44.10	55.40	53.80	57.20	57.55	<u>58.20</u>	**61.00**
	8–10	22.40	20.35	22.65	36.20	32.70	37.60	38.10	<u>40.05</u>	**43.45**
ACP-Hard	0–2	88.20	91.20	83.10	91.60	91.80	92.40	92.70	<u>92.90</u>	**93.50**
	2–4	53.00	56.60	55.90	60.70	59.80	62.10	62.50	<u>62.90</u>	**64.20**
	4–6	34.00	36.00	36.70	44.50	42.80	45.80	46.20	<u>46.60</u>	**48.90**
	6–8	20.50	22.80	23.40	26.00	24.80	26.90	27.10	<u>27.40</u>	**30.20**
	8–10	7.10	7.60	7.80	7.50	6.80	7.90	8.00	<u>8.15</u>	**11.55**
BoardgameQA	0–2	86.10	85.80	86.20	84.00	84.70	<u>86.60</u>	86.50	86.30	**87.20**
	2–4	79.00	79.40	80.60	69.50	75.10	<u>81.20</u>	81.05	80.75	**82.50**
	4–6	72.60	74.20	75.20	60.70	68.20	<u>75.70</u>	75.60	75.10	**78.00**
	6–8	67.60	69.00	69.60	53.10	61.50	69.20	<u>69.60</u>	68.80	**72.40**
	8–10	59.70	63.80	64.10	40.60	51.70	62.50	<u>62.75</u>	62.30	**66.15**
MuSR	0–2	79.70	82.40	83.40	78.90	80.60	83.50	<u>83.70</u>	82.90	**84.30**
	2–4	68.20	78.80	79.60	64.10	70.80	79.75	<u>79.90</u>	77.80	**81.20**
	4–6	59.80	73.80	74.70	55.30	64.20	74.85	<u>75.00</u>	72.70	**77.30**
	6–8	51.60	69.20	69.60	48.90	58.60	69.55	<u>69.80</u>	67.10	**72.60**
	8–10	41.60	61.80	62.20	36.10	45.80	62.45	<u>62.59</u>	56.90	**65.99**
Natural Plan	0–2	68.20	61.50	62.80	80.40	77.20	81.20	<u>81.80</u>	80.90	**82.40**
	2–4	45.50	47.20	48.60	57.40	54.30	60.80	<u>61.40</u>	60.90	**62.70**
	4–6	32.00	36.10	34.20	43.20	38.60	44.80	<u>45.50</u>	45.10	**47.80**
	6–8	18.70	23.40	20.10	28.90	25.30	29.80	30.20	<u>30.60</u>	**33.40**
	8–10	4.30	6.70	5.20	13.10	10.40	13.70	<u>14.45</u>	14.20	**17.85**

Table 15: Complete complexity-stratified results corresponding to Figure 3. All values are accuracy scores in percentage. Examples are divided into five equal-width bins according to normalized complexity scores from 0 to 10. Best results are in bold and second-best results are underlined.

Method	ACPBench Hard	BoardgameQA	MuSR	Natural Plan	Avg.
AgendaFlow	**48.67** (±0.69)	**76.25** (±0.46)	**75.28** (±0.48)	**47.83** (±0.66)	**62.01** (±0.35)
Generic	44.75 (±0.95)	70.17 (±0.88)	67.22 (±0.79)	42.42 (±0.91)	56.14 (±0.47)
Indep.-Role	45.60 (±0.86)	73.40 (±0.71)	72.40 (±0.66)	44.20 (±0.82)	58.90 (±0.41)
– Role	<u>46.92</u> (±0.72)	<u>75.83</u> (±0.53)	<u>74.83</u> (±0.56)	<u>46.25</u> (±0.68)	<u>60.96</u> (±0.37)
– Routing	45.83 (±0.80)	74.67 (±0.62)	73.94 (±0.59)	44.75 (±0.76)	59.80 (±0.42)
– Moderator	43.17 (±1.05)	71.58 (±0.94)	69.89 (±0.86)	41.92 (±1.00)	56.64 (±0.52)

Table 16: Complete ablation results with standard deviations. All values are accuracy scores in percentage, reported as the mean over five independent runs with different random seeds. Standard deviations are shown in parentheses. "Generic" denotes the protocol-free multi-agent variant. "Indep.-Role" denotes an independent role ensemble that uses the same instantiated agents and role profiles as AgendaFlow, but each agent solves the full task independently without observing other agents or participating in phase-level discussion. "– Role", "– Routing", and "– Moderator" denote removing functional role design, task-adaptive protocol routing, and moderator control from AgendaFlow. Best results are in bold and second-best results are underlined.

Backbone	Method	Natural Plan	ACPBench Hard	BoardgameQA	Avg.
Kimi2	Self-Consistency	34.90 (±0.62)	36.83 (±0.58)	75.58 (±0.41)	49.10 (±0.32)
	AutoGen	40.60 (±1.02)	42.83 (±1.08)	68.25 (±1.00)	50.56 (±0.60)
	MAgICoRe	<u>47.40</u> (±0.74)	<u>45.75</u> (±0.72)	<u>76.05</u> (±0.48)	<u>56.40</u> (±0.38)
	AgendaFlow	**47.80** (±0.66)	**48.67** (±0.69)	**76.25** (±0.46)	**57.57** (±0.36)
GPT-5.5	Self-Consistency	45.00 (±0.64)	45.80 (±0.61)	**81.00** (±0.42)	57.27 (±0.33)
	AutoGen	48.20 (±1.04)	49.90 (±1.01)	73.50 (±0.96)	57.20 (±0.58)
	MAgICoRe	<u>52.70</u> (±0.72)	<u>52.85</u> (±0.70)	79.10 (±0.47)	<u>61.55</u> (±0.37)
	AgendaFlow	**53.70** (±0.64)	**54.80** (±0.67)	<u>79.70</u> (±0.48)	**62.73** (±0.34)
DeepSeek-V4-Pro	Self-Consistency	40.00 (±0.63)	42.50 (±0.59)	<u>77.50</u> (±0.42)	53.33 (±0.32)
	AutoGen	44.10 (±1.05)	47.30 (±1.04)	69.80 (±0.98)	53.73 (±0.59)
	MAgICoRe	<u>49.20</u> (±0.73)	<u>49.90</u> (±0.71)	76.90 (±0.48)	<u>58.67</u> (±0.37)
	AgendaFlow	**50.50** (±0.65)	**51.90** (±0.68)	**77.60** (±0.49)	**60.00** (±0.35)
Qwen3.6	Self-Consistency	43.80 (±0.64)	48.00 (±0.61)	<u>80.20</u> (±0.43)	57.33 (±0.33)
	AutoGen	47.00 (±1.03)	51.20 (±1.00)	72.60 (±0.95)	56.93 (±0.57)
	MAgICoRe	<u>51.00</u> (±0.71)	<u>53.95</u> (±0.69)	78.80 (±0.47)	<u>61.25</u> (±0.36)
	AgendaFlow	**52.20** (±0.63)	**56.30** (±0.66)	**80.40** (±0.47)	**62.97** (±0.33)
GLM5.1	Self-Consistency	42.60 (±0.63)	42.80 (±0.60)	<u>81.80</u> (±0.41)	55.73 (±0.32)
	AutoGen	46.20 (±1.04)	47.80 (±1.02)	74.10 (±0.96)	56.03 (±0.58)
	MAgICoRe	<u>50.00</u> (±0.72)	<u>49.90</u> (±0.70)	80.05 (±0.46)	<u>59.98</u> (±0.36)
	AgendaFlow	**50.80** (±0.64)	**52.00** (±0.67)	**81.90** (±0.46)	**61.57** (±0.34)
Average	Self-Consistency	41.26 (±0.28)	43.19 (±0.27)	**79.22** (±0.19)	54.55 (±0.14)
	AutoGen	45.22 (±0.47)	47.81 (±0.46)	71.65 (±0.43)	54.89 (±0.27)
	MAgICoRe	<u>50.06</u> (±0.32)	<u>50.47</u> (±0.31)	78.18 (±0.21)	<u>59.57</u> (±0.16)
	AgendaFlow	**51.00** (±0.37)	**52.73** (±0.39)	<u>79.17</u> (±0.27)	**60.97** (±0.20)

Table 17: Backbone generalization results across five backbone models. All values are accuracy scores in percentage, reported as the mean over five independent runs with standard deviations in parentheses. Natural Plan, ACPBench Hard, and BoardgameQA are used as representative benchmarks for the generalization analysis. Best results within each backbone group are in bold and second-best results are underlined.

Method	Tokens	ACPBench Hard	BoardgameQA	Natural Plan	Avg.
Self-Consistency	18.1K	37.6 (±0.55)	76.0 (±0.39)	36.7 (±0.61)	50.1 (±0.31)
AutoGen	18.0K	44.4 (±1.15)	69.9 (±1.04)	43.5 (±1.06)	52.6 (±0.62)
MAgICoRe	18.2K	<u>46.8</u> (±0.72)	<u>76.1</u> (±0.48)	<u>46.9</u> (±0.74)	<u>56.6</u> (±0.38)
AgendaFlow	18.3K	**49.3** (±0.68)	**76.5** (±0.45)	**48.8** (±0.67)	**58.2** (±0.36)

Table 18: Complete token-matched results with standard deviations. All values except token counts are accuracy scores in percentage, reported as the mean over five independent runs with different random seeds. Standard deviations are shown in parentheses. Baselines are scaled by increasing self-consistency samples, AutoGen interaction turns, or refinement iterations until their average token usage approaches that of AgendaFlow. The average is computed over ACPBench Hard, BoardgameQA, and Natural Plan. Best results are in bold and second-best results are underlined.